DOCKER

Complete Mastery Guide

Dockerfile • Docker Compose • Buildx • Kaniko • Admin • Interview Q&A

📦 **Core Docker**	⚙️ **Dockerfile**	🔧 **Compose**	🚀 **Buildx & Kaniko**

🐳 SECTION 1: DOCKER FUNDAMENTALS

Architecture, Core Concepts, Installation & Lifecycle

1. What is Docker?

Docker is an open-source platform that automates the deployment, scaling, and management of applications inside lightweight, portable containers. It packages an application and all its dependencies into a standardized unit called a container, ensuring consistent behavior across development, testing, and production environments.

1.1 Key Concepts

Concept	Description	Analogy
Image	Read-only template used to create containers. Contains OS, app code, dependencies.	Blueprint / Class
Container	Running instance of an image. Isolated process with its own filesystem, network, and process space.	House built from blueprint
Registry	Storage and distribution system for Docker images (Docker Hub, ECR, GCR, ACR).	App Store
Dockerfile	Text file with instructions to build a Docker image automatically.	Recipe / Build Script
Volume	Persistent storage mechanism that exists outside container lifecycle.	External Hard Drive
Network	Virtual network allowing containers to communicate with each other and outside world.	LAN / VPN
Layer	Each instruction in a Dockerfile creates a layer. Layers are cached and shared.	Incremental backup
Docker Daemon	Background service (dockerd) managing containers, images, networks, volumes.	Engine
Docker CLI	Command-line interface to interact with Docker daemon.	Remote Control
Docker Engine	Client-server application: daemon + REST API + CLI.	Complete System

1.2 Docker Architecture

📄 Docker Architecture Diagram

```
┌──────────────────────────────────────────────────────────┐
│                       DOCKER CLIENT                      │
│           docker build / run / pull / push               │
```

```
    └──────────────────────────┬──────────────────────────────────────┘
                               │
                               │ REST API / Unix Socket
    ┌──────────────────────────▼──────────────────────────────────────┐
    │                   DOCKER DAEMON (dockerd)                       │
    │   ┌──────────┐   ┌──────────┐   ┌──────────┐                   │
    │   │ Containers│   │  Images  │   │ Volumes  │   Networks        │
    │   └──────────┘   └──────────┘   └──────────┘                   │
    └─────────────────────────────────────────────────────────────────┘
                               │
    ┌──────────────────────────▼──────────────────────────────────────┐
    │                CONTAINER RUNTIME (containerd)                   │
    │          runc → actual Linux containers (namespaces)            │
    └─────────────────────────────────────────────────────────────────┘
```

1.3 Docker vs Virtual Machines

Feature	Docker Containers	Virtual Machines
OS	Shares host OS kernel	Full guest OS per VM
Startup	Seconds (milliseconds)	Minutes
Size	MBs	GBs
Performance	Near-native	Overhead from hypervisor
Isolation	Process-level (namespaces)	Hardware-level (hypervisor)
Portability	High — image runs anywhere	Limited — tied to hypervisor
Density	Hundreds per host	Tens per host

1.4 Installation

◆ Install Docker on Ubuntu/Debian

📄 **Ubuntu/Debian Installation**

```
# Remove old versions
sudo apt-get remove docker docker-engine docker.io containerd runc

# Update and install prerequisites
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg lsb-release

# Add Docker's GPG key
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg

# Add Docker repository
echo "deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | \
```

```
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Install Docker Engine
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin

# Start and enable Docker
sudo systemctl start docker
sudo systemctl enable docker

# Add user to docker group (avoid sudo)
sudo usermod -aG docker $USER
newgrp docker

# Verify installation
docker --version
docker info
```

◆ Install Docker on CentOS/RHEL

```
📄 CentOS/RHEL Installation
sudo yum install -y yum-utils
sudo yum-config-manager --add-repo https://download.docker.com/linux/centos/docker-
ce.repo
sudo yum install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
compose-plugin
sudo systemctl start docker && sudo systemctl enable docker
```

2. Essential Docker Commands

2.1 Image Commands

```
📄 Image Management Commands
# Pull an image from registry
docker pull nginx:latest
docker pull ubuntu:22.04
docker pull python:3.11-slim

# List local images
docker images
docker image ls
docker images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}'

# Inspect image details
docker inspect nginx:latest
docker image inspect nginx --format='{{.Config.Env}}'
```

```
# Remove image
docker rmi nginx:latest
docker image rm nginx:latest
docker rmi $(docker images -q)          # Remove ALL images
docker image prune                       # Remove dangling images
docker image prune -a                    # Remove all unused images

# Tag an image
docker tag nginx:latest myrepo/nginx:v1.0

# Push image to registry
docker push myrepo/nginx:v1.0

# Save/Load image as tar
docker save nginx:latest > nginx.tar
docker save -o nginx.tar nginx:latest
docker load < nginx.tar
docker load -i nginx.tar

# Image history (layers)
docker history nginx:latest
docker history --no-trunc nginx:latest
```

2.2 Container Commands

📄 **Container Management Commands**

```
# Run a container
docker run nginx                            # Foreground
docker run -d nginx                         # Detached (background)
docker run -it ubuntu bash                  # Interactive terminal
docker run --name mycontainer nginx         # Named container
docker run -p 8080:80 nginx                 # Port mapping (host:container)
docker run -p 127.0.0.1:8080:80 nginx       # Bind to specific IP
docker run -v /host/path:/container/path nginx # Volume mount
docker run -v myvolume:/app/data nginx       # Named volume
docker run -e MY_VAR=value nginx             # Environment variable
docker run --env-file .env nginx             # Env from file
docker run --rm nginx                        # Auto-remove on stop
docker run --network mynet nginx             # Custom network
docker run --memory=512m nginx               # Memory limit
docker run --cpus=1.5 nginx                  # CPU limit
docker run --restart=always nginx            # Restart policy
docker run --user 1000:1000 nginx            # Run as specific user
docker run --read-only nginx                 # Read-only filesystem

# List containers
docker ps                                 # Running containers
```

```
docker ps -a                              # All containers (inc. stopped)
docker ps -q                              # Only IDs
docker ps --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'

# Start/Stop/Restart
docker start mycontainer
docker stop mycontainer                   # Graceful (SIGTERM)
docker kill mycontainer                   # Forceful (SIGKILL)
docker restart mycontainer
docker pause mycontainer
docker unpause mycontainer

# Remove containers
docker rm mycontainer                     # Remove stopped container
docker rm -f mycontainer                  # Force remove running
docker rm $(docker ps -aq)                # Remove ALL containers
docker container prune                    # Remove all stopped containers

# Inspect container
docker inspect mycontainer
docker inspect --format='{{.NetworkSettings.IPAddress}}' mycontainer

# Execute command in running container
docker exec mycontainer ls /app
docker exec -it mycontainer bash          # Interactive shell
docker exec -u root mycontainer whoami    # As root

# Copy files
docker cp mycontainer:/app/file.txt ./file.txt
docker cp ./file.txt mycontainer:/app/file.txt

# View logs
docker logs mycontainer
docker logs -f mycontainer                # Follow (tail -f)
docker logs --tail 100 mycontainer        # Last 100 lines
docker logs --since 2024-01-01 mycontainer
docker logs --timestamps mycontainer

# Stats / resource usage
docker stats
docker stats mycontainer --no-stream
docker top mycontainer                    # Running processes

# Container diff (changed files)
docker diff mycontainer

# Commit container to image
```

```
docker commit mycontainer mynewimage:v1.0

# Port mappings
docker port mycontainer
```

SECTION 2: DOCKERFILE

Instructions, Best Practices, Multi-Stage & Real-World Examples

3. Dockerfile Deep Dive

A Dockerfile is a text document containing instructions to assemble a Docker image. Each instruction creates a new layer in the image. Understanding Dockerfile instructions and best practices is critical for building efficient, secure, and maintainable images.

3.1 All Dockerfile Instructions

Instruction	Purpose	Example
FROM	Base image — must be first instruction (except ARG)	FROM ubuntu:22.04
RUN	Execute commands during build (creates new layer)	RUN apt-get install -y curl
CMD	Default command when container starts (can be overridden)	CMD ["node", "app.js"]
ENTRYPOINT	Fixed command that always runs (CMD becomes args)	ENTRYPOINT ["python"]
COPY	Copy files from host to image	COPY . /app
ADD	Like COPY but supports URLs and tar extraction	ADD app.tar.gz /app
WORKDIR	Set working directory for subsequent instructions	WORKDIR /app
ENV	Set environment variables (available at runtime)	ENV NODE_ENV=production
ARG	Build-time variables (not available at runtime)	ARG VERSION=1.0
EXPOSE	Documents port container listens on (metadata only)	EXPOSE 8080
VOLUME	Creates mount point for external volumes	VOLUME ['/data']
USER	Set user/group for RUN, CMD, ENTRYPOINT	USER appuser:appgroup
LABEL	Add metadata key-value pairs to image	LABEL version='1.0'
HEALTHCHECK	Command to check container health	HEALTHCHECK CMD curl -f http://localhost/
SHELL	Override default shell for RUN commands	SHELL ["/bin/bash", "-c"]
STOPSIGNAL	Signal sent to container to stop	STOPSIGNAL SIGTERM
ONBUILD	Trigger instruction when used as base image	ONBUILD RUN npm install

3.2 CMD vs ENTRYPOINT

📄 CMD vs ENTRYPOINT

```
# CMD — can be overridden at 'docker run'
CMD ["node", "app.js"]
# docker run myimage                  → runs: node app.js
# docker run myimage python server.py → runs: python server.py (CMD ignored)

# ENTRYPOINT — always runs, CMD becomes args
ENTRYPOINT ["python"]
CMD ["app.py"]
# docker run myimage                  → runs: python app.py
# docker run myimage server.py        → runs: python server.py

# Shell form vs Exec form
RUN apt-get update                    # Shell form → /bin/sh -c 'apt-get update'
CMD ["nginx", "-g", "daemon off;"]    # Exec form → PREFERRED (no shell wrapper)

# Shell form does NOT receive Unix signals properly!
# Always use exec form for CMD and ENTRYPOINT
```

3.3 Production-Grade Node.js Dockerfile

📄 Node.js Multi-Stage Dockerfile

```
# ── Stage 1: Dependencies ──────────────────────────────
FROM node:20-alpine AS deps
WORKDIR /app

# Copy only package files first (better layer caching)
COPY package.json package-lock.json ./
RUN npm ci --only=production && npm cache clean --force

# ── Stage 2: Build ─────────────────────────────────────
FROM node:20-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build

# ── Stage 3: Production ────────────────────────────────
FROM node:20-alpine AS production

# Security: Create non-root user
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

WORKDIR /app
```

```
# Copy only what's needed
COPY --from=deps /app/node_modules ./node_modules
COPY --from=build /app/dist ./dist
COPY package.json ./

# Set ownership
RUN chown -R appuser:appgroup /app
USER appuser

ENV NODE_ENV=production
ENV PORT=3000
EXPOSE 3000

HEALTHCHECK --interval=30s --timeout=10s --start-period=30s --retries=3 \
  CMD wget -qO- http://localhost:3000/health || exit 1

ENTRYPOINT ["node"]
CMD ["dist/server.js"]
```

3.4 Production-Grade Python Dockerfile

📄 **Python FastAPI Dockerfile**

```
FROM python:3.11-slim AS base

# Set environment variables
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1

# ── Build stage ──────────────────────────────────────────
FROM base AS builder
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential gcc && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --user -r requirements.txt

# ── Production stage ─────────────────────────────────────
FROM base AS production
WORKDIR /app

# Non-root user
RUN groupadd -r appgroup && useradd -r -g appgroup appuser

# Copy installed packages from builder
```

```
COPY --from=builder /root/.local /home/appuser/.local
COPY --chown=appuser:appgroup . .

USER appuser
ENV PATH=/home/appuser/.local/bin:$PATH

EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

3.5 Java/Spring Boot Dockerfile

Spring Boot Dockerfile

```
FROM eclipse-temurin:17-jdk-alpine AS build
WORKDIR /app
COPY gradlew .
COPY gradle gradle
COPY build.gradle settings.gradle .
COPY src src
RUN ./gradlew build -x test

FROM eclipse-temurin:17-jre-alpine AS production
WORKDIR /app
RUN addgroup -S spring && adduser -S spring -G spring
COPY --from=build /app/build/libs/*.jar app.jar
RUN chown spring:spring app.jar
USER spring
EXPOSE 8080
HEALTHCHECK CMD wget -qO- http://localhost:8080/actuator/health || exit 1
ENTRYPOINT ["java", "-jar", "app.jar"]
```

3.6 Dockerfile Best Practices

- Use official minimal base images (alpine, slim, distroless) to reduce attack surface
- Use multi-stage builds to keep final images small — build tools not included in production
- Order instructions from least to most frequently changing for optimal layer caching
- Combine RUN commands with && to reduce layers: RUN apt-get update && apt-get install -y pkg && rm -rf /var/lib/apt/lists/*
- Always use a .dockerignore file to exclude unnecessary files
- Never hardcode secrets — use build args, environment variables, or secrets
- Run containers as non-root users for security
- Pin specific image versions (never use 'latest' in production)
- Use COPY instead of ADD unless you need URL fetch or tar extraction
- Add HEALTHCHECK for production containers
- Use exec form (JSON array) for CMD and ENTRYPOINT

3.7 .dockerignore File

📄 .dockerignore

```
# .dockerignore — prevents these from being sent to build context
.git
.gitignore
.dockerignore
node_modules
npm-debug.log
Dockerfile
docker-compose*.yml
.env
.env.*
*.md
dist
coverage
*.test.js
.nyc_output
__pycache__
*.pyc
.pytest_cache
*.log
tmp/
.DS_Store
```

🔧 SECTION 3: DOCKER COMPOSE

Multi-Container Applications, Networking, Volumes & Real-World Stacks

4. Docker Compose

Docker Compose is a tool for defining and running multi-container Docker applications. You use a YAML file to configure all your application's services, networks, and volumes, then start everything with a single command.

4.1 Docker Compose Commands

📄 Docker Compose Commands

```
# Start services (build if needed)
docker compose up
docker compose up -d                        # Detached mode
docker compose up --build                   # Force rebuild images
docker compose up --build --no-deps web     # Only rebuild 'web' service
docker compose up --scale web=3             # Scale web service to 3 replicas

# Stop services
docker compose down
docker compose down -v                      # Also remove volumes
docker compose down --rmi all               # Also remove images
docker compose stop                         # Stop without removing

# View status
docker compose ps
docker compose ps -a
docker compose top

# Logs
docker compose logs
docker compose logs -f
docker compose logs -f web                  # Specific service
docker compose logs --tail=50 web

# Execute commands
docker compose exec web bash
docker compose exec -T web node -e 'console.log(process.env)'
docker compose run web npm test             # One-off command (new container)
docker compose run --rm web npm test        # Remove after run

# Build
docker compose build
docker compose build web                    # Build specific service
```

```
docker compose build --no-cache            # No cache

# Pull latest images
docker compose pull

# Config validation
docker compose config
docker compose config --services

# Restart
docker compose restart
docker compose restart web

# Pause/Unpause
docker compose pause web
docker compose unpause web

# Specify compose file
docker compose -f docker-compose.prod.yml up -d
docker compose -f docker-compose.yml -f docker-compose.override.yml up
```

4.2 Complete docker-compose.yml Reference

📄 **Complete docker-compose.yml**

```
version: '3.9'

# ── Named Volumes ──────────────────────────────────────
volumes:
  postgres_data:
    driver: local
  redis_data:
  app_uploads:
    driver_opts:
      type: none
      o: bind
      device: /data/uploads

# ── Networks ───────────────────────────────────────────
networks:
  frontend:
    driver: bridge
  backend:
    driver: bridge
    internal: true          # No external access
  monitoring:

# ── Secrets ────────────────────────────────────────────
```

```yaml
secrets:
  db_password:
    file: ./secrets/db_password.txt
  jwt_secret:
    external: true          # Managed outside compose

services:

  # ── API Service ──────────────────────────────────────
  api:
    build:
      context: ./api
      dockerfile: Dockerfile
      target: production
      args:
        NODE_ENV: production
        BUILD_DATE: '2024-01-01'
    image: myapp/api:latest
    container_name: myapp_api
    restart: unless-stopped
    ports:
      - '3000:3000'
    environment:
      - NODE_ENV=production
      - DB_HOST=postgres
      - REDIS_URL=redis://redis:6379
    env_file:
      - .env
      - .env.production
    secrets:
      - db_password
    volumes:
      - app_uploads:/app/uploads
      - ./logs:/app/logs
    networks:
      - frontend
      - backend
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_started
    healthcheck:
      test: ['CMD', 'wget', '-qO-', 'http://localhost:3000/health']
      interval: 30s
      timeout: 10s
      retries: 3
```

```
      start_period: 40s
    deploy:
      resources:
        limits:
          cpus: '0.5'
          memory: 512M
        reservations:
          cpus: '0.25'
          memory: 256M
    logging:
      driver: json-file
      options:
        max-size: '10m'
        max-file: '3'
    labels:
      - 'traefik.enable=true'
      - 'com.example.service=api'

  # ── PostgreSQL ──────────────────────────────────────
  postgres:
    image: postgres:15-alpine
    container_name: myapp_postgres
    restart: unless-stopped
    environment:
      POSTGRES_DB: myapp
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets:
      - db_password
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./init-db:/docker-entrypoint-initdb.d:ro
    networks:
      - backend
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U myuser -d myapp']
      interval: 10s
      timeout: 5s
      retries: 5

  # ── Redis ───────────────────────────────────────────
  redis:
    image: redis:7-alpine
    container_name: myapp_redis
    restart: unless-stopped
    command: redis-server --appendonly yes --requirepass ${REDIS_PASSWORD}
    volumes:
```

```
      - redis_data:/data
    networks:
      - backend

  # ── Nginx Reverse Proxy ──────────────────────────────
  nginx:
    image: nginx:alpine
    container_name: myapp_nginx
    restart: unless-stopped
    ports:
      - '80:80'
      - '443:443'
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
      - ./nginx/ssl:/etc/nginx/ssl:ro
    networks:
      - frontend
    depends_on:
      - api
```

4.3 Environment-Specific Compose Files

📄 **Multi-environment Compose**

```
# docker-compose.yml (base)
services:
  web:
    image: myapp:latest
    environment:
      - NODE_ENV=development

# docker-compose.override.yml (auto-loaded in dev)
services:
  web:
    build: .
    volumes:
      - .:/app          # Live reload
    ports:
      - '3000:3000'

# docker-compose.prod.yml
services:
  web:
    restart: always
    environment:
      - NODE_ENV=production
    deploy:
      replicas: 3
```

```
# Usage:
docker compose up                                          # Uses base + override
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d
```

🚀 SECTION 4: DOCKER BUILDX

Multi-Platform Builds, BuildKit, Advanced Caching

5. Docker Buildx

Docker Buildx is a CLI plugin extending Docker build capabilities with full support for BuildKit features including multi-platform builds, build secrets, SSH mounts, and advanced caching strategies.

5.1 Buildx Setup & Builder Management

📄 Buildx Setup

```
# Check buildx version
docker buildx version

# List builders
docker buildx ls

# Create a new builder instance
docker buildx create --name mybuilder --driver docker-container
docker buildx create --name mybuilder --driver docker-container --use

# Create builder with specific platforms
docker buildx create --name mybuilder \
  --driver docker-container \
  --platform linux/amd64,linux/arm64,linux/arm/v7 \
  --use

# Bootstrap the builder (pull buildkitd image)
docker buildx inspect --bootstrap
docker buildx inspect mybuilder --bootstrap

# Switch between builders
docker buildx use mybuilder
docker buildx use default

# Remove a builder
docker buildx rm mybuilder

# Create builder for Kubernetes
docker buildx create \
  --name kube-builder \
  --driver kubernetes \
  --driver-opt namespace=buildkit
```

```
# Enable QEMU for cross-platform emulation
docker run --privileged --rm tonistiigi/binfmt --install all
```

5.2 Multi-Platform Builds

📄 **Multi-Platform Build Commands**

```
# Build for single platform
docker buildx build --platform linux/amd64 -t myapp:amd64 .

# Build for multiple platforms simultaneously
docker buildx build \
  --platform linux/amd64,linux/arm64,linux/arm/v7 \
  -t myrepo/myapp:latest \
  --push .

# Build and load to local Docker (single platform only)
docker buildx build --platform linux/amd64 -t myapp:latest --load .

# Build multi-platform and push to registry
docker buildx build \
  --platform linux/amd64,linux/arm64 \
  -t myrepo/myapp:1.0 \
  -t myrepo/myapp:latest \
  --push .

# Build with multiple tags
docker buildx build \
  --platform linux/amd64,linux/arm64 \
  -t myrepo/myapp:1.2.3 \
  -t myrepo/myapp:1.2 \
  -t myrepo/myapp:1 \
  -t myrepo/myapp:latest \
  --push .

# Build with specific Dockerfile
docker buildx build \
  --platform linux/amd64,linux/arm64 \
  -f Dockerfile.prod \
  -t myrepo/myapp:latest \
  --push .

# Inspect multi-platform manifest
docker buildx imagetools inspect myrepo/myapp:latest
```

5.3 Buildx Advanced Caching

📄 **Buildx Caching Strategies**

```
# ── Registry Cache ──────────────────────────────────────
# Cache to registry (best for CI/CD)
docker buildx build \
  --cache-from type=registry,ref=myrepo/myapp:buildcache \
  --cache-to type=registry,ref=myrepo/myapp:buildcache,mode=max \
  -t myrepo/myapp:latest \
  --push .

# ── Local Cache ─────────────────────────────────────────
docker buildx build \
  --cache-from type=local,src=/tmp/buildcache \
  --cache-to type=local,dest=/tmp/buildcache,mode=max \
  -t myapp:latest .

# ── Inline Cache (embed in image) ───────────────────────
docker buildx build \
  --cache-from type=registry,ref=myrepo/myapp:latest \
  --cache-to type=inline \
  -t myrepo/myapp:latest \
  --push .

# ── GitHub Actions Cache ────────────────────────────────
docker buildx build \
  --cache-from type=gha \
  --cache-to type=gha,mode=max \
  -t myrepo/myapp:latest .

# mode=max → caches ALL intermediate layers (best cache hit rate)
# mode=min → caches only final image layers (smaller cache)

# ── S3 Cache ────────────────────────────────────────────
docker buildx build \
  --cache-from type=s3,region=us-east-1,bucket=my-build-cache \
  --cache-to type=s3,region=us-east-1,bucket=my-build-cache,mode=max \
  -t myrepo/myapp:latest .
```

5.4 Buildx Build Secrets & SSH

📄 **Secrets & SSH**

```
# Build with secrets (never stored in image layers!)
docker buildx build \
  --secret id=mysecret,src=./secret.txt \
  -t myapp:latest .
```

```
# In Dockerfile — use secret:
# RUN --mount=type=secret,id=mysecret \
#     cat /run/secrets/mysecret

# SSH forwarding for private repos
docker buildx build \
  --ssh default=$SSH_AUTH_SOCK \
  -t myapp:latest .

# In Dockerfile:
# RUN --mount=type=ssh git clone git@github.com:private/repo.git

# Build args
docker buildx build \
  --build-arg NODE_ENV=production \
  --build-arg BUILD_DATE=$(date -u +%Y-%m-%dT%H:%M:%SZ) \
  -t myapp:latest .

# Bake — define builds in HCL/JSON file
docker buildx bake -f docker-bake.hcl --push
```

5.5 Bake File Example (docker-bake.hcl)

docker-bake.hcl

```
variable "TAG" {
  default = "latest"
}

variable "REGISTRY" {
  default = "myrepo"
}

group "default" {
  targets = ["api", "worker", "frontend"]
}

target "api" {
  context = "./api"
  dockerfile = "Dockerfile"
  platforms = ["linux/amd64", "linux/arm64"]
  tags = [
    "${REGISTRY}/api:${TAG}",
    "${REGISTRY}/api:latest"
  ]
  cache-from = [
    "type=registry,ref=${REGISTRY}/api:buildcache"
  ]
```

```
  cache-to = [
    "type=registry,ref=${REGISTRY}/api:buildcache,mode=max"
  ]
}

target "worker" {
  context = "./worker"
  inherits = ["api"]
  args = {
    SERVICE = "worker"
  }
}
```

☁️ SECTION 5: KANIKO

Daemonless Builds in Kubernetes, CI/CD Integration

6. Kaniko — Daemonless Docker Builds

Kaniko is a tool to build container images from a Dockerfile, inside a container or Kubernetes cluster. Unlike Docker, Kaniko doesn't require a Docker daemon — it executes each command in the Dockerfile completely in userspace. This makes it safe for building images in environments where you cannot run a privileged Docker daemon (like Kubernetes pods).

6.1 Why Kaniko?

Aspect	Docker-in-Docker (DinD)	Kaniko
Privileges	Requires --privileged (security risk)	No privileged mode needed
Daemon	Full Docker daemon required	No daemon — userspace only
Kubernetes	Complex setup, security concerns	Native Kubernetes support
Caching	Limited without registry	Registry-based layer caching
Speed	Fast (native daemon)	Slightly slower (userspace layers)
Security	High risk — root access	Low risk — rootless builds

6.2 Kaniko Build Context Sources

📄 Kaniko Context Sources

```
# ── Context from GCS ──────────────────────────────
docker run -it \
  -v $(pwd):/workspace \
  gcr.io/kaniko-project/executor:latest \
  --context gs://my-bucket/build-context.tar.gz \
  --dockerfile /workspace/Dockerfile \
  --destination myrepo/myapp:latest

# ── Context from S3 ───────────────────────────────
docker run -it \
  -e AWS_ACCESS_KEY_ID=$AWS_ACCESS_KEY_ID \
  -e AWS_SECRET_ACCESS_KEY=$AWS_SECRET_ACCESS_KEY \
  gcr.io/kaniko-project/executor:latest \
  --context s3://my-bucket/build-context.tar.gz \
  --destination myrepo/myapp:latest

# ── Context from local directory ──────────────────
docker run -it \
  -v $(pwd):/workspace \
```

```
  gcr.io/kaniko-project/executor:latest \
  --context dir:///workspace \
  --dockerfile /workspace/Dockerfile \
  --destination myrepo/myapp:latest

# ── Context from Git ──────────────────────────────────────
docker run -it \
  gcr.io/kaniko-project/executor:latest \
  --context 'git://github.com/user/repo#refs/heads/main' \
  --destination myrepo/myapp:latest
```

6.3 Kaniko in Kubernetes

📄 **Kubernetes Pod Manifest**

```
# ── kaniko-pod.yaml ──────────────────────────────────────
apiVersion: v1
kind: Pod
metadata:
  name: kaniko-build
  namespace: ci
spec:
  restartPolicy: Never
  volumes:
    - name: kaniko-secret
      secret:
        secretName: regcred         # Docker registry credentials
    - name: dockerfile-cm
      configMap:
        name: dockerfile-config
  containers:
    - name: kaniko
      image: gcr.io/kaniko-project/executor:latest
      args:
        - '--context=git://github.com/user/repo#main'
        - '--dockerfile=Dockerfile'
        - '--destination=myrepo/myapp:latest'
        - '--destination=myrepo/myapp:1.0.0'
        - '--cache=true'
        - '--cache-repo=myrepo/myapp/cache'
        - '--compressed-caching=false'
        - '--snapshot-mode=redo'
      volumeMounts:
        - name: kaniko-secret
          mountPath: /kaniko/.docker
      resources:
        limits:
          memory: '2Gi'
```

```
            cpu: '1000m'
```

6.4 Kaniko in GitLab CI/CD

GitLab CI/CD with Kaniko

```
# .gitlab-ci.yml
stages:
  - build
  - deploy

build-image:
  stage: build
  image:
    name: gcr.io/kaniko-project/executor:debug
    entrypoint: ['']
  script:
    - mkdir -p /kaniko/.docker
    - echo "{\"auths\":{\"$CI_REGISTRY\":{\"auth\":\"$(printf '%s:%s'
      '$CI_REGISTRY_USER' '$CI_REGISTRY_PASSWORD' | base64 | tr -d '\n')\"}}}"
      > /kaniko/.docker/config.json
    - /kaniko/executor
      --context $CI_PROJECT_DIR
      --dockerfile $CI_PROJECT_DIR/Dockerfile
      --destination $CI_REGISTRY_IMAGE:$CI_COMMIT_TAG
      --destination $CI_REGISTRY_IMAGE:latest
      --cache=true
      --cache-repo=$CI_REGISTRY_IMAGE/cache
  rules:
    - if: $CI_COMMIT_TAG
```

6.5 Kaniko in GitHub Actions

GitHub Actions with Kaniko

```
# .github/workflows/build.yml
name: Build and Push Docker Image

on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

```

```
      - name: Set up Kaniko config
        run: |
          mkdir -p ~/.docker
          echo '{"auths":{"https://index.docker.io/v1/":{"auth":"'$(echo -n
            '${{ secrets.DOCKERHUB_USERNAME }}:${{ secrets.DOCKERHUB_TOKEN }}'
            '| base64)'"}}}' > ~/.docker/config.json

      - name: Build with Kaniko
        uses: docker://gcr.io/kaniko-project/executor:latest
        with:
          args: >-
            --context=dir:///github/workspace
            --dockerfile=Dockerfile
            --destination=myrepo/myapp:latest
            --destination=myrepo/myapp:${{ github.sha }}
            --cache=true
```

6.6 Kaniko Key Flags

Flag	Description	Example
--context	Build context location (dir, GCS, S3, git)	--context dir:///workspace
--dockerfile	Path to Dockerfile	--dockerfile /workspace/Dockerfile
--destination	Image to push (registry/image:tag)	--destination myrepo/app:v1
--cache	Enable layer caching	--cache=true
--cache-repo	Repository to store cache	--cache-repo myrepo/cache
--no-push	Build but don't push	--no-push
--tarPath	Path to save image as tar	--tarPath /out/image.tar
--build-arg	Build arguments	--build-arg ENV=prod
--target	Multi-stage build target	--target production
--skip-tls-verify	Skip TLS for private registry	--skip-tls-verify
--snapshot-mode	How to snapshot filesystem (redo/full/time)	--snapshot-mode=redo
--verbosity	Log level (panic/fatal/error/warn/info/debug)	--verbosity=info

🌐 SECTION 6: DOCKER NETWORKING & VOLUMES

Network Types, DNS, Volume Management & Storage Drivers

7. Docker Networking

7.1 Network Types

Network Driver	Description	Use Case
bridge	Default. Creates isolated network on host. Containers on same bridge can communicate.	Single host, multi-container apps
host	Container shares host network stack. No isolation. Best performance.	High-performance, single container
none	Disables all networking for the container.	Maximum isolation, batch jobs
overlay	Multi-host networking for Swarm/Kubernetes.	Distributed apps, microservices
macvlan	Assigns MAC address to container — appears as physical device on network.	Legacy apps needing direct LAN access
ipvlan	Like macvlan but shares MAC. Better for environments restricting multiple MACs.	Cloud VMs with MAC restrictions

7.2 Network Commands

📄 Network Commands

```
# List networks
docker network ls

# Create network
docker network create mynet
docker network create --driver bridge mynet
docker network create --driver overlay myoverlay
docker network create --subnet=192.168.1.0/24 --gateway=192.168.1.1 mynet
docker network create --internal mynet         # No external access

# Inspect network
docker network inspect mynet
docker network inspect bridge

# Connect/Disconnect container
docker network connect mynet mycontainer
```

```
docker network connect --ip 192.168.1.10 mynet mycontainer
docker network disconnect mynet mycontainer

# Remove network
docker network rm mynet
docker network prune                          # Remove unused networks

# Run container in specific network
docker run --network mynet --name web nginx
docker run --network host nginx               # Host network
docker run --network none nginx               # No network

# DNS resolution — containers can ping by name on same network
docker run --network mynet --name app1 nginx
docker run --network mynet --name app2 ubuntu ping app1  # Works!
```

8. Docker Volumes

8.1 Volume Types

Type	Description	When to Use
Named Volume	Managed by Docker. Stored in /var/lib/docker/volumes/	Persistent data: databases, user uploads
Bind Mount	Mounts host directory into container. Full host path required.	Development: live code reloading
tmpfs Mount	Stored only in host memory. Not persisted.	Sensitive data, temp files
Anonymous Volume	Created by Docker automatically without a name.	Temporary data, discarded on container removal

8.2 Volume Commands

📄 Volume Commands

```
# Create named volume
docker volume create mydata
docker volume create --driver local \
  --opt type=nfs \
  --opt o=addr=192.168.1.100,rw \
  --opt device=:/path/to/dir \
  nfs-volume

# List volumes
docker volume ls
docker volume ls --filter dangling=true

# Inspect volume
```

```
docker volume inspect mydata

# Remove volumes
docker volume rm mydata
docker volume prune                          # Remove unused volumes
docker volume prune --filter label=env=dev

# Use volumes
docker run -v mydata:/app/data nginx           # Named volume
docker run -v $(pwd):/app nginx                # Bind mount
docker run -v $(pwd)/src:/app/src:ro nginx     # Read-only bind
docker run --mount source=mydata,target=/data nginx   # Mount syntax
docker run --tmpfs /tmp nginx                  # tmpfs

# Backup volume data
docker run --rm \
  -v mydata:/source:ro \
  -v $(pwd):/backup \
  ubuntu tar czf /backup/mydata_backup.tar.gz -C /source .

# Restore volume data
docker run --rm \
  -v mydata:/target \
  -v $(pwd):/backup:ro \
  ubuntu tar xzf /backup/mydata_backup.tar.gz -C /target
```

📦 SECTION 7: REGISTRY & SECURITY

Private Registry, Authentication, Security Hardening

9. Docker Registry

9.1 Working with Registries

📄 **Registry Authentication**

```
# Login to Docker Hub
docker login
docker login -u username -p password

# Login to AWS ECR
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS --password-stdin 123456789.dkr.ecr.us-east-1.amazonaws.com

# Login to GCR
gcloud auth configure-docker
gcloud auth print-access-token | docker login -u oauth2accesstoken --password-stdin gcr.io

# Login to Azure ACR
az acr login --name myacr

# Tag for different registries
docker tag myapp:latest docker.io/myuser/myapp:latest       # Docker Hub
docker tag myapp:latest 123.dkr.ecr.us-east-1.amazonaws.com/myapp:latest  # ECR
docker tag myapp:latest gcr.io/myproject/myapp:latest       # GCR
docker tag myapp:latest myacr.azurecr.io/myapp:latest       # ACR

# Push images
docker push myuser/myapp:latest

# Logout
docker logout
docker logout myacr.azurecr.io
```

9.2 Private Registry Setup

📄 **Private Registry**

```
# Run local registry
docker run -d -p 5000:5000 \
  --name registry \
  --restart=always \
```

```
  -v registry_data:/var/lib/registry \
  -v /path/to/certs:/certs \
  -e REGISTRY_HTTP_TLS_CERTIFICATE=/certs/domain.crt \
  -e REGISTRY_HTTP_TLS_KEY=/certs/domain.key \
  registry:2

# Push to local registry
docker tag myapp:latest localhost:5000/myapp:latest
docker push localhost:5000/myapp:latest
docker pull localhost:5000/myapp:latest

# Registry with authentication
docker run -d -p 5000:5000 \
  -v /auth:/auth \
  -e REGISTRY_AUTH=htpasswd \
  -e REGISTRY_AUTH_HTPASSWD_PATH=/auth/htpasswd \
  -e REGISTRY_AUTH_HTPASSWD_REALM='Registry Realm' \
  registry:2

# Create htpasswd file
htpasswd -Bc /auth/htpasswd username
```

10. Docker Security

10.1 Security Best Practices

- Run containers as non-root users — use USER instruction in Dockerfile
- Use official and verified base images from trusted sources
- Scan images for vulnerabilities with docker scout, trivy, or snyk
- Use read-only filesystems where possible (--read-only flag)
- Limit capabilities — drop all caps, add only required ones
- Use secrets management — never store secrets in images or env vars in plain text
- Enable content trust — sign images with Docker Content Trust (DCT)
- Restrict network access — use internal networks, disable inter-container communication when not needed
- Set resource limits on all containers (--memory, --cpus)
- Use AppArmor or seccomp profiles to restrict syscalls

10.2 Security Commands

📄 Security Commands

```
# Scan image for vulnerabilities
docker scout cves myapp:latest
docker scout quickview myapp:latest

# Run with limited capabilities
docker run --cap-drop ALL --cap-add NET_BIND_SERVICE nginx
```

```

# Read-only filesystem
docker run --read-only --tmpfs /tmp nginx

# No new privileges
docker run --security-opt no-new-privileges nginx

# Custom seccomp profile
docker run --security-opt seccomp=/path/to/seccomp.json nginx

# Enable Docker Content Trust
export DOCKER_CONTENT_TRUST=1
docker pull myapp:latest  # Verifies signature

# View container capabilities
docker inspect --format='{{.HostConfig.CapAdd}} {{.HostConfig.CapDrop}}'
mycontainer

# Check running processes
docker top mycontainer

# Limit PID count
docker run --pids-limit 100 nginx

# User namespace remapping (in daemon.json)
# { 'userns-remap': 'default' }
```

🛠️ SECTION 8: DOCKER ADMINISTRATION

Daemon Config, System Management, Monitoring & Troubleshooting

11. Docker Admin & System Management

11.1 Docker Daemon Configuration

📄 /etc/docker/daemon.json

```
# Docker daemon config: /etc/docker/daemon.json
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "100m",
    "max-file": "3"
  },
  "storage-driver": "overlay2",
  "data-root": "/var/lib/docker",
  "max-concurrent-downloads": 10,
  "max-concurrent-uploads": 5,
  "default-address-pools": [
    {"base": "172.30.0.0/16", "size": 24}
  ],
  "dns": ["8.8.8.8", "8.8.4.4"],
  "insecure-registries": ["localhost:5000"],
  "registry-mirrors": ["https://mirror.gcr.io"],
  "live-restore": true,
  "userland-proxy": false,
  "metrics-addr": "0.0.0.0:9323",
  "experimental": true
}

# Reload daemon config (no restart needed)
sudo kill -SIGHUP $(pidof dockerd)
sudo systemctl reload docker

# Restart daemon
sudo systemctl restart docker
sudo systemctl status docker

# Check daemon logs
sudo journalctl -u docker -f
sudo journalctl -u docker --since '1 hour ago'
```

11.2 System Cleanup & Maintenance

System Cleanup Commands

```
# System overview
docker system df                                # Disk usage
docker system df -v                             # Verbose disk usage
docker system info                              # System-wide info

# Prune commands (free up space)
docker system prune                             # Remove all unused resources
docker system prune -a                          # Include unused images
docker system prune -a --volumes                # Include volumes
docker system prune -a --volumes -f             # Force (no prompt)

docker container prune                          # Stopped containers
docker image prune                              # Dangling images
docker image prune -a                           # All unused images
docker volume prune                             # Unused volumes
docker network prune                            # Unused networks
docker builder prune                            # Build cache
docker builder prune -a                         # All build cache
docker buildx prune -a                          # Buildx cache

# Check disk usage
du -sh /var/lib/docker/*
du -sh /var/lib/docker/overlay2

# System events
docker events
docker events --since '24h'
docker events --filter event=start
docker events --filter container=myapp
```

11.3 Monitoring & Performance

Monitoring Commands

```
# Real-time stats
docker stats
docker stats --no-stream                        # Snapshot
docker stats --format 'table {{.Container}}\t{{.CPUPerc}}\t{{.MemUsage}}'

# Container resource details
docker inspect --format='{{json .HostConfig.Resources}}' mycontainer | python3 -m
json.tool

# Top processes in container
docker top mycontainer
docker top mycontainer aux
```

```
# Get container IP
docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}'
mycontainer

# Port bindings
docker inspect -f '{{json .NetworkSettings.Ports}}' mycontainer

# Health status
docker inspect --format='{{.State.Health.Status}}' mycontainer
docker inspect --format='{{json .State.Health}}' mycontainer | python3 -m json.tool

# Check container exit code
docker inspect --format='{{.State.ExitCode}}' mycontainer

# Prometheus metrics (if enabled in daemon.json)
curl http://localhost:9323/metrics

# cAdvisor for container monitoring
docker run -d -p 8080:8080 \
  --name cadvisor \
  -v /:/rootfs:ro \
  -v /var/run:/var/run:ro \
  -v /sys:/sys:ro \
  -v /var/lib/docker/:/var/lib/docker:ro \
  gcr.io/cadvisor/cadvisor:latest
```

11.4 Docker Swarm (Orchestration)

📄 **Docker Swarm Commands**

```
# Initialize Swarm
docker swarm init
docker swarm init --advertise-addr 192.168.1.100

# Join Swarm
docker swarm join --token SWMTKN-... 192.168.1.100:2377
docker swarm join-token worker   # Print worker join command
docker swarm join-token manager  # Print manager join command

# Node management
docker node ls
docker node inspect mynode
docker node update --availability drain mynode  # Drain before maintenance
docker node update --availability active mynode
docker node promote mynode      # Worker → Manager
docker node demote mynode       # Manager → Worker
docker node rm mynode
```

```
# Service management
docker service create --name web --replicas 3 -p 80:80 nginx
docker service ls
docker service inspect web
docker service ps web
docker service logs web -f
docker service scale web=5
docker service update --image nginx:1.25 web
docker service update --update-parallelism 2 --update-delay 10s web
docker service rollback web
docker service rm web

# Stack deploy (compose in Swarm mode)
docker stack deploy -c docker-compose.yml mystack
docker stack ls
docker stack services mystack
docker stack ps mystack
docker stack rm mystack

# Leave Swarm
docker swarm leave
docker swarm leave --force    # On manager
```

11.5 Troubleshooting

📄 **Troubleshooting Commands**

```
# Container won't start — check logs
docker logs --tail 100 mycontainer

# Check exit reason
docker inspect --format='{{.State.Error}}' mycontainer
docker inspect --format='{{.State.ExitCode}}' mycontainer

# Debug with shell in failed container
docker run --entrypoint sh -it myimage
docker run --entrypoint bash -it myimage

# Attach to running container
docker attach mycontainer       # Careful! Ctrl+C may kill container

# Port not accessible
docker port mycontainer
netstat -tlnp | grep 8080

# DNS resolution issues
docker run --rm busybox nslookup google.com
```

```
docker run --rm --network mynet busybox nslookup myservice

# Container can't reach internet
# Check iptables rules
sudo iptables -L -n -v | grep DOCKER

# Storage issues
docker system df
df -h /var/lib/docker

# OOMKilled — out of memory
docker inspect --format='{{.State.OOMKilled}}' mycontainer

# Permission denied in container
docker run --user 0 myimage   # Run as root to debug
docker exec -u root mycontainer bash

# Network connectivity between containers
docker exec -it container1 ping container2
docker exec -it container1 curl http://container2:8080/health

# View all container environment variables
docker exec mycontainer env

# strace in container
docker run --cap-add SYS_PTRACE --pid=host myimage
```

11.6 Useful Docker Inspect Queries

```
📄 Inspect Format Queries
# Get container IP address
docker inspect -f '{{.NetworkSettings.IPAddress}}' c1

# Get all network info
docker inspect -f '{{json .NetworkSettings.Networks}}' c1 | python3 -m json.tool

# Get all environment variables
docker inspect -f '{{range .Config.Env}}{{println .}}{{end}}' c1

# Get mount points
docker inspect -f '{{range .Mounts}}{{.Source}} → {{.Destination}}{{println}}
{{end}}' c1

# Get restart policy
docker inspect -f '{{.HostConfig.RestartPolicy.Name}}' c1

# Get all labels
```

```
docker inspect -f '{{json .Config.Labels}}' c1

# Get exposed ports
docker inspect -f '{{json .Config.ExposedPorts}}' c1

# Get running state
docker inspect -f 'Status: {{.State.Status}}, Running: {{.State.Running}}' c1

# Get image layers
docker history --no-trunc myimage | head -20
```

🔄 SECTION 9: REAL-WORLD CI/CD WORKFLOWS

GitHub Actions, GitLab, Jenkins & Production Patterns

12. Real-World Workflows

12.1 GitHub Actions — Complete Docker CI/CD

📄 GitHub Actions CI/CD Pipeline

```yaml
# .github/workflows/docker-ci.yml
name: Docker Build, Test, and Deploy

on:
  push:
    branches: [main, develop]
    tags: ['v*.*.*']
  pull_request:
    branches: [main]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run tests
        run: docker compose -f docker-compose.test.yml up --abort-on-container-exit

  build:
    needs: test
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v4

      - name: Set up QEMU
        uses: docker/setup-qemu-action@v3

      - name: Set up Docker Buildx
```

```
        uses: docker/setup-buildx-action@v3

      - name: Login to GHCR
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Extract metadata
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}
          tags: |
            type=ref,event=branch
            type=ref,event=pr
            type=semver,pattern={{version}}
            type=semver,pattern={{major}}.{{minor}}

      - name: Build and push
        uses: docker/build-push-action@v5
        with:
          context: .
          platforms: linux/amd64,linux/arm64
          push: ${{ github.event_name != 'pull_request' }}
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

12.2 Jenkins Pipeline

Jenkinsfile

```
// Jenkinsfile
pipeline {
  agent any
  environment {
    REGISTRY = 'myrepo'
    IMAGE = "${REGISTRY}/myapp"
    TAG = "${BUILD_NUMBER}"
    DOCKER_CREDENTIALS = credentials('docker-hub-creds')
  }
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Test') {
      steps {
```

```
        sh 'docker compose -f docker-compose.test.yml up --abort-on-container-exit'
      }
      post { always {
        sh 'docker compose -f docker-compose.test.yml down -v'
      }}
    }
    stage('Build') {
      steps {
        sh """
          docker buildx build \\
            --platform linux/amd64,linux/arm64 \\
            --cache-from type=registry,ref=${IMAGE}:cache \\
            --cache-to type=registry,ref=${IMAGE}:cache,mode=max \\
            -t ${IMAGE}:${TAG} \\
            -t ${IMAGE}:latest \\
            --push .
        """
      }
    }
    stage('Security Scan') {
      steps {
        sh "trivy image --exit-code 1 --severity HIGH,CRITICAL ${IMAGE}:${TAG}"
      }
    }
    stage('Deploy') {
      when { branch 'main' }
      steps {
        sh "docker service update --image ${IMAGE}:${TAG} myapp_web"
      }
    }
  }
}
```

🎯 SECTION 10: INTERVIEW QUESTIONS & ANSWERS

Beginner → Intermediate → Advanced → Scenario-Based

13. Interview Q&A

13.1 Beginner Level

Q: What is Docker and why do we use it?
A: Docker is a containerization platform that packages applications and their dependencies into isolated containers. We use it for consistency across environments (dev/test/prod), faster deployments, efficient resource use, and microservices architecture.

Q: What is the difference between an image and a container?
A: An image is a read-only template (like a class/blueprint) containing the OS, application code, and dependencies. A container is a running instance of an image (like an object) — it's an isolated process with its own filesystem, network, and process space.

Q: What is Docker Hub?
A: Docker Hub is the default public registry for Docker images. It hosts official images (nginx, ubuntu, postgres), community images, and private repositories. Alternatives include Amazon ECR, Google GCR, Azure ACR, and self-hosted registries.

Q: What is a Dockerfile?
A: A Dockerfile is a text file with instructions (FROM, RUN, COPY, CMD, etc.) used to automatically build Docker images. Each instruction creates a layer in the image. You build an image from it using 'docker build'.

Q: What is the difference between CMD and ENTRYPOINT?
A: CMD provides a default command/arguments that can be completely overridden at runtime. ENTRYPOINT defines the fixed executable that always runs, with CMD providing default arguments. Best practice: use ENTRYPOINT for the command, CMD for default arguments. Both should use exec form (JSON array).

Q: What is a Docker volume and why is it needed?
A: Volumes are persistent storage that exists outside the container lifecycle. They're needed because container filesystems are ephemeral — data is lost when a container is removed. Volumes allow data persistence, sharing between containers, and host-container file sharing.

13.2 Intermediate Level

Q: What is a multi-stage Docker build and why use it?
A: Multi-stage builds use multiple FROM statements in one Dockerfile. Earlier stages compile/build the application; the final stage copies only the built artifacts. This produces much smaller production images (often 10x smaller) because build tools, compilers, and intermediate files are discarded.

Q: Explain Docker networking. What are the different network drivers?
A: Docker provides several network drivers: bridge (default, isolated network on single host), host (shares host network, no isolation), none (no networking), overlay (multi-host Swarm networking), macvlan (container gets physical network MAC address). Containers on the same user-defined bridge network can communicate by container name using Docker's embedded DNS.

Q: How do you handle secrets in Docker?
A: Best practices: use Docker Secrets (Swarm), mount secrets as files via --secret flag in buildx builds, use environment variables from .env files (not committed to git), integrate with HashiCorp Vault or cloud secret managers (AWS Secrets Manager, GCP Secret Manager). Never hardcode secrets in Dockerfiles or images.

Q: What is Docker Compose and what is it used for?
A: Docker Compose is a tool for defining and running multi-container applications using a YAML file. It manages service dependencies, networks, volumes, and environment configuration. 'docker compose up' starts the entire application stack. It's ideal for development environments and simple production deployments.

Q: How does Docker layer caching work? How do you optimize it?
A: Each Dockerfile instruction creates a layer. If an instruction hasn't changed, Docker reuses the cached layer. Optimization: put least-changing instructions first (FROM, RUN apt-get), copy only package files before the full source (COPY package.json . then RUN npm install then COPY . .), use --no-cache to force rebuild, use BuildKit for advanced caching.

Q: What is the difference between COPY and ADD in Dockerfile?
A: COPY simply copies files/directories from build context to image. ADD does everything COPY does plus: supports URLs as source, and automatically extracts tar archives. Best practice: use COPY for regular files; use ADD only when you need tar extraction or URL fetching.

13.3 Advanced Level

Q: What is Docker Buildx and how does it differ from regular docker build?
A: Docker Buildx extends docker build with BuildKit features: multi-platform builds (build for amd64 AND arm64 simultaneously), advanced caching (registry, S3, GHA), build secrets without layer exposure, SSH forwarding, bake files for complex build graphs, and multiple output types. It uses a builder instance that can run in docker-container or kubernetes driver.

Q: What is Kaniko and when would you use it over Docker-in-Docker?
A: Kaniko builds container images from a Dockerfile inside a container or Kubernetes pod without requiring a Docker daemon. Use Kaniko in Kubernetes CI/CD when you can't run privileged containers (security requirement), need rootless builds, want to avoid DinD complexity, or are in managed Kubernetes environments (GKE, EKS) where DinD is restricted.

Q: How do you implement zero-downtime deployments with Docker?
A: Strategies: (1) Docker Swarm rolling updates with --update-parallelism and --update-delay; (2) Blue-Green deployment — run new version alongside old, switch traffic at load balancer; (3) Canary releases — route small % of traffic to new version; (4) Health checks ensure new containers are ready before old ones are removed. Always configure HEALTHCHECK in Dockerfile.

Q: How would you optimize a Docker image that is 2GB in size?
A: Steps: Use multi-stage builds to separate build/runtime; switch to slim/alpine base images; combine RUN commands and clean up apt/yum caches in same layer; remove development dependencies; use .dockerignore;

analyze with 'docker history' and 'dive' tool; consider distroless images for production. A well-optimized image should be 50-200MB.

Q: What are Docker namespaces and cgroups?

A: Namespaces provide isolation: pid (process isolation), net (network isolation), ipc (inter-process communication), mnt (filesystem mounts), uts (hostname), user (user IDs). cgroups (control groups) limit and account for resource usage: CPU, memory, disk I/O, network bandwidth. Together they form the foundation of container isolation.

Q: Explain Docker Content Trust (DCT). How does image signing work?

A: DCT uses Notary to provide cryptographic signing and verification of Docker images. Publishers sign images with their private key; consumers verify with the public key. Enable with: export DOCKER_CONTENT_TRUST=1. Uses The Update Framework (TUF). Prevents pulling tampered/unverified images. Important for supply chain security.

13.4 Scenario-Based Questions

Q: Your container keeps restarting with exit code 137. What do you do?

A: Exit code 137 means OOMKilled (Out of Memory Killed). Check: docker inspect --format='{{.State.OOMKilled}}' container → will show true. Solutions: (1) Increase memory limit with --memory flag; (2) Optimize application memory usage; (3) Check for memory leaks with docker stats; (4) Use JVM flags for Java apps (-Xmx); (5) Enable swap limit in Docker daemon.

Q: A container can access the internet but not another container in the same compose file. What do you check?

A: Check: (1) Both containers on same network (docker network inspect); (2) Using service name (not container name) for DNS; (3) No typos in service name reference; (4) depends_on is set correctly; (5) Target container is actually running (docker compose ps); (6) No 'internal: true' on the network; (7) Check with docker exec ping servicename from the container.

Q: How do you update a running service with zero downtime in Docker Swarm?

A: docker service update --image newimage:v2 --update-parallelism 1 --update-delay 30s --update-failure-action rollback myservice. This updates one replica at a time, waits 30 seconds between updates, and automatically rolls back if update fails. Monitor with 'docker service ps myservice'. Set health check on container so Swarm waits for healthy status.

Q: How do you reduce build times in a CI/CD pipeline?

A: Strategies: (1) Registry-based layer cache with buildx --cache-from/--cache-to; (2) Optimize Dockerfile layer order (copy package files before source); (3) Use GitHub Actions cache type; (4) Multi-stage builds — only rebuild changed stages; (5) Use BuildKit parallel stage execution; (6) Pin base image digests for consistent pulls; (7) Use kaniko with --cache=true for Kubernetes builds.

📋 SECTION 11: QUICK REFERENCE CHEAT SHEET

Essential Commands at a Glance

14. Docker Cheat Sheet

14.1 Most Used Commands

Task	Command	Notes
Build image	docker build -t name:tag .	Use --no-cache to force rebuild
Build multi-platform	docker buildx build --platform linux/amd64,linux/arm64 --push -t repo/img:tag .	Requires buildx builder
Run container	docker run -d -p 8080:80 --name c1 img:tag	-d detached, -p port map
Shell into container	docker exec -it c1 bash	Use sh if bash not available
View logs	docker logs -f c1	-f to follow, --tail N for last N lines
Stop all containers	docker stop $(docker ps -q)	Graceful stop all running
Remove everything	docker system prune -a --volumes -f	WARNING: removes all unused resources
Compose up	docker compose up -d --build	-d detached, --build rebuild
Compose down+volumes	docker compose down -v	Also removes named volumes
Inspect IP	docker inspect -f '{{.NetworkSettings.IPAddress}}' c1	Get container IP address
Copy from container	docker cp c1:/app/file.txt .	Copy file to current dir
Resource stats	docker stats --no-stream	One-time resource snapshot
Push to registry	docker push repo/image:tag	Must be logged in first
Scan vulnerabilities	docker scout cves image:tag	Requires Docker Scout
Prune build cache	docker builder prune -a	Free up buildx cache space

14.2 Dockerfile Quick Reference

Instruction	Syntax	Example
FROM	FROM image:tag [AS name]	FROM node:20-alpine AS build
RUN	RUN command1 && command2	RUN apt-get update && apt-get install -y curl
COPY	COPY src dest	COPY package*.json ./
WORKDIR	WORKDIR /path	WORKDIR /app
ENV	ENV KEY=VALUE	ENV NODE_ENV=production
ARG	ARG NAME[=default]	ARG VERSION=1.0
EXPOSE	EXPOSE port	EXPOSE 3000

Instruction	Syntax	Example
CMD	CMD ["exec","arg1"]	CMD ["node", "server.js"]
ENTRYPOINT	ENTRYPOINT ["exec"]	ENTRYPOINT ["python"]
HEALTHCHECK	HEALTHCHECK CMD cmd	HEALTHCHECK CMD curl -f http://localhost/
USER	USER user:group	USER appuser:appgroup
VOLUME	VOLUME /path	VOLUME /data

14.3 Restart Policies

Policy	Behavior	Use Case
no (default)	Never automatically restart	Short-lived tasks, batch jobs
always	Always restart regardless of exit code	Critical services in production
on-failure	Restart only on non-zero exit	Apps that should stop on success
unless-stopped	Restart unless manually stopped or Docker stops	Most production services (recommended)

14.4 Common Exit Codes

Exit Code	Meaning	Action
0	Success — container exited normally	Expected for batch jobs
1	Application error — app crashed	Check application logs
125	Docker daemon error	Check Docker daemon status
126	Command cannot be invoked (permission)	Check ENTRYPOINT/CMD permissions
127	Command not found in container	Check if binary exists in image
130	Container stopped by Ctrl+C (SIGINT)	Normal interactive session exit
137	OOMKilled — out of memory	Increase memory limit, check leaks
143	Graceful stop — SIGTERM received	Normal stop via docker stop
255	Exit status out of range — general error	Check container logs for details

💡 PRO TIPS: Use 'docker system df' regularly to monitor disk usage. Always add HEALTHCHECK to production Dockerfiles. Use 'unless-stopped' restart policy for production services. Pin image versions (never :latest in production). Use multi-stage builds for all compiled languages. Leverage BuildKit cache mounts for package managers: RUN --mount=type=cache,target=/root/.npm npm install

🐳 End of Docker Complete Mastery Guide 🐳